

MẪU ĐỒ ÁN ĐẠI HỌC

Đề tài:

Sinh viên: Chu Tuấn Nghĩa

GVHD: Cao Tuấn Dũng

Năm: 2026

Mục lục

I	Mở đầu	2
1	Bối cảnh nghiên cứu	2
2	Vấn đề nghiên cứu	3
3	Phương pháp nghiên cứu	3
4	Cấu trúc báo cáo	4
II	Cơ sở lý thuyết và các công nghệ liên quan	5
1	Large Language Model Agent và Agent Memory	5
2	Retrieval-Augmented Generation	7
3	Knowledge Graph trong hệ thống memory	8
4	Personalized PageRank	8
III	Tổng quan các nghiên cứu liên quan	11
IV	Phân tích bài toán và thiết kế phương pháp đề xuất	12
V	Thực nghiệm và đánh giá	13
VI	Kết luận và hướng phát triển	14

Chương I

Mở đầu

1 Bối cảnh nghiên cứu

Trong những năm gần đây, các mô hình ngôn ngữ lớn, hay Large Language Models, đã đạt được nhiều thành tựu nổi bật trong các tác vụ xử lý ngôn ngữ tự nhiên như hỏi đáp, tóm tắt văn bản, sinh nội dung, lập luận và hỗ trợ ra quyết định. Trên nền tảng đó, các hệ thống LLM Agent ngày càng được quan tâm nhờ khả năng tương tác với người dùng, sử dụng công cụ, lập kế hoạch và thực hiện các tác vụ phức tạp trong nhiều ngữ cảnh khác nhau.

Tuy nhiên, một hạn chế lớn của các LLM Agent hiện nay là khả năng ghi nhớ dài hạn còn chưa hiệu quả. Phần lớn mô hình ngôn ngữ chỉ có thể xử lý thông tin trong một cửa sổ ngữ cảnh giới hạn. Khi cuộc hội thoại kéo dài qua nhiều phiên, các thông tin quan trọng từ những tương tác trước có thể bị mất, bị lãng quên hoặc không được truy xuất chính xác. Điều này làm giảm khả năng duy trì tính nhất quán, cá nhân hóa và suy luận dựa trên lịch sử tương tác của hệ thống.

Để giải quyết vấn đề này, nhiều nghiên cứu đã đề xuất các kiến trúc bộ nhớ ngoài cho LLM Agent, trong đó Retrieval-Augmented Generation là một hướng tiếp cận phổ biến. Thay vì chỉ dựa vào tri thức đã được huấn luyện trong tham số mô hình, hệ thống có thể lưu trữ lịch sử hội thoại hoặc tri thức bên ngoài, sau đó truy xuất các thông tin liên quan để hỗ trợ quá trình sinh câu trả lời. Tuy nhiên, các phương pháp RAG truyền thống thường dựa chủ yếu vào vector similarity, nên có thể gặp khó khăn khi cần kết nối nhiều mẫu thông tin rời rạc, đặc biệt trong các tác vụ multi-hop reasoning hoặc temporal reasoning.

Gần đây, các phương pháp kết hợp Knowledge Graph với RAG đã được nghiên cứu nhằm tổ chức bộ nhớ theo dạng có cấu trúc hơn. Thay vì lưu trữ thông tin dưới dạng các đoạn văn bản độc lập, hệ thống có thể biểu diễn tri thức dưới dạng các thực thể, quan

hệ và triples. Cách tiếp cận này giúp mô hình khai thác được mối liên hệ giữa các thông tin trong quá khứ, từ đó cải thiện khả năng truy xuất và suy luận dài hạn.

2 Vấn đề nghiên cứu

LiCoMemory là một framework đáng chú ý khi đề xuất CogniGraph, một kiến trúc đồ thị phân cấp nhẹ cho bộ nhớ dài hạn của Agent. LiCoMemory tổ chức thông tin theo nhiều tầng, bao gồm session summary, entity-relation graph và original dialogue chunks, nhằm hỗ trợ cập nhật và truy xuất bộ nhớ theo thời gian thực. Theo bài báo, mục tiêu của LiCoMemory là giảm độ dư thừa của graph, tăng hiệu quả truy xuất và cải thiện khả năng suy luận trong các hội thoại dài hạn.

Tuy nhiên, tầng entity-relation trong LiCoMemory vẫn còn tiềm năng để cải tiến. Việc truy xuất triples nếu chỉ dựa vào matching trực tiếp giữa query và triples có thể bỏ sót các thông tin liên quan gián tiếp. Trong nhiều trường hợp, câu hỏi của người dùng cần nhiều evidence nằm rải rác trong graph, không phải lúc nào cũng khớp trực tiếp với entity hoặc relation được nêu trong query.

Mục tiêu của nghiên cứu là sẽ thử nghiệm LiCoMemory tích hợp với thuật toán duyệt đồ thị, cụ thể ở đây là Personalized PageRank (PPR), từ đó đánh giá được tác động của nó đối với kiến trúc CogniGraph tổng thể.

3 Phương pháp nghiên cứu

Để thực hiện đề tài, báo cáo sử dụng kết hợp phương pháp nghiên cứu lý thuyết, phân tích hệ thống, thiết kế thuật toán và thực nghiệm đánh giá.

Trước hết, đề tài tiến hành nghiên cứu các công trình liên quan đến long-term memory cho LLM Agent, bao gồm LiCoMemory, HippoRAG, HippoRAG 2 và CatRAG. Việc phân tích các công trình này giúp xác định các hướng tiếp cận hiện có, ưu điểm, hạn chế và khoảng trống nghiên cứu.

Tiếp theo, đề tài phân tích kiến trúc gốc của LiCoMemory để hiểu rõ cách hệ thống tổ chức bộ nhớ, xây dựng CogniGraph, trích xuất triples và truy xuất thông tin. Từ quá trình phân tích này, đề tài xác định vấn đề chính nằm ở việc entity-relation retrieval có thể chưa khai thác đầy đủ khả năng liên kết trên graph.

Sau đó, nghiên cứu đề xuất phương pháp cải tiến bằng Personalized PageRank. Cụ thể, query của người dùng được xử lý để xác định các seed nodes hoặc seed triples. Từ các seed này, thuật toán PPR được thực hiện trên entity-relation graph để lan truyền độ

liên quan và tìm ra các thực thể, quan hệ hoặc triples có khả năng hỗ trợ trả lời câu hỏi.

Cuối cùng, thực hiện đánh giá thực nghiệm bằng cách so sánh phương pháp đề xuất với các biến thể baseline. Các chỉ số đánh giá bao gồm độ chính xác câu trả lời, recall của evidence, chất lượng retrieved triples, latency và token cost.

4 Cấu trúc báo cáo

Chương 1 trình bày phần mở đầu, bao gồm bối cảnh nghiên cứu, bối cảnh nghiên cứu và phương pháp thực hiện.

Chương 2 trình bày cơ sở lý thuyết liên quan đến LLM Agent, long-term memory, Retrieval-Augmented Generation, Knowledge Graph, Personalized PageRank và các chỉ số đánh giá retrieval.

Chương 3 giới thiệu và phân tích các nghiên cứu liên quan, bao gồm LiCoMemory, HippoRAG, HippoRAG 2 và CatRAG. Chương này cũng chỉ ra khoảng trống nghiên cứu mà đề tài hướng đến.

Chương 4 trình bày phân tích bài toán và phương pháp đề xuất, trong đó tập trung vào việc cải tiến LiCoMemory bằng Personalized PageRank trên entity-relation graph.

Chương 5 tiến hành thực nghiệm và đánh giá trên 2 bộ dữ liệu LOCOMO và LongmemEval để so sánh với baseline.

Chương 6 kết luận lại bài nghiên cứu, chỉ ra những điểm cải tiến, hạn chế và từ đó đề xuất thêm các hướng khả thi trong tương lai.

Chương II

Cơ sở lý thuyết và các công nghệ liên quan

1 Large Language Model Agent và Agent Memory

LARGE LANGUAGE MODEL

Large Language Model là mô hình ngôn ngữ lớn được huấn luyện trên khối lượng dữ liệu văn bản rất lớn để hiểu và sinh ngôn ngữ tự nhiên. Các mô hình này có thể thực hiện nhiều tác vụ như trả lời câu hỏi, tóm tắt văn bản, dịch máy, sinh nội dung và hỗ trợ lập luận. Nhờ khả năng xử lý ngữ cảnh tốt, Large Language Model trở thành nền tảng quan trọng cho nhiều hệ thống trí tuệ nhân tạo hiện đại. Tuy nhiên, bản thân mô hình vẫn bị giới hạn bởi cửa sổ ngữ cảnh và không thể tự ghi nhớ toàn bộ lịch sử tương tác dài hạn.

AI AGENT

AI Agent là một hệ thống dựa trên mô hình ngôn ngữ lớn, được hình thành từ bốn mô-đun cốt lõi:

- **Mô-đun nhận thức (Perception Module):** có nhiệm vụ cảm nhận môi trường và chuyển đổi các quan sát bên ngoài thành các biểu diễn nội bộ mà hệ thống có thể xử lý.
- **Mô-đun suy luận (Reasoning Module):** có nhiệm vụ phân rã các nhiệm vụ phức tạp, suy luận về sự phụ thuộc giữa các bước, tương tác với bộ nhớ và xây dựng chiến lược thực thi.
- **Hệ thống bộ nhớ (Memory System):** bao gồm bộ nhớ ngắn hạn phục vụ cho quá trình suy luận tức thời và bộ nhớ dài hạn dùng để lưu giữ kinh nghiệm, hỗ trợ cho quá trình suy luận trong tương lai.

- **Mô-đun hành động (Action Module):** có nhiệm vụ thực thi các hành động trong môi trường.

Mục tiêu của AI Agent là tối đa hóa các chỉ số đánh giá mong muốn, chẳng hạn như độ chính xác, tỷ lệ hoàn thành nhiệm vụ và phần thưởng.

AGENT MEMORY

Bộ nhớ đã trở thành một thành phần quan trọng trong việc phát triển các LLM Agent hướng tới bốn mục tiêu chính.

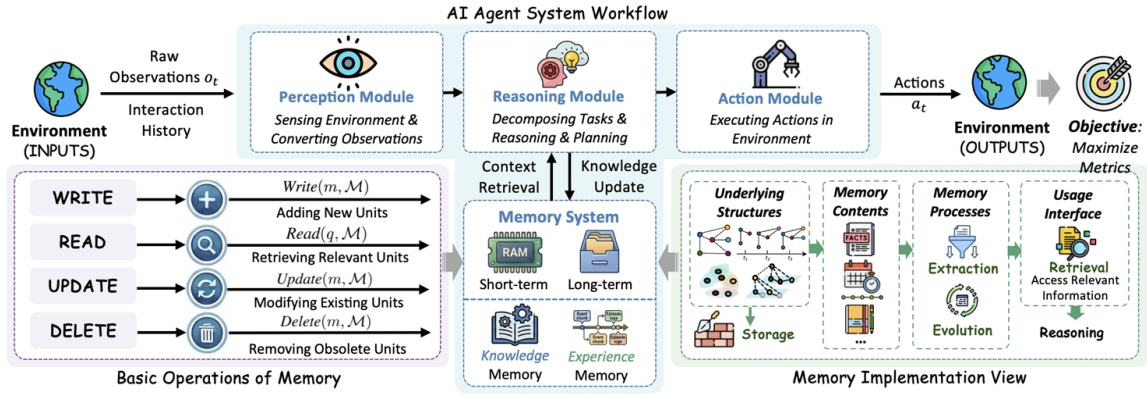
Thứ nhất, **cá nhân hóa và đặc tả ngữ cảnh**. Bộ nhớ cho phép Agent ghi nhận sở thích của người dùng, lịch sử tương tác và ngữ cảnh đặc thù của từng nhiệm vụ để tạo ra phản hồi phù hợp hơn. Ví dụ, trong kỹ thuật phần mềm, Agent có thể ghi nhớ thói quen làm việc của người dùng; trong hội thoại, Agent có thể ghi nhớ phong cách giao tiếp mà người dùng mong muốn. Nhờ đó, bộ nhớ đóng vai trò kết nối tri thức tổng quát của mô hình với ngữ cảnh cụ thể của từng người dùng, giúp phản hồi trở nên cá nhân hóa và phù hợp hơn.

Thứ hai, **hỗ trợ suy luận dài hạn vượt ra ngoài cửa sổ ngữ cảnh**. Các mô hình ngôn ngữ lớn thường bị giới hạn bởi cửa sổ ngữ cảnh hữu hạn và tri thức tham số cố định. Trong khi đó, hệ thống bộ nhớ cung cấp một kho lưu trữ bên ngoài có khả năng mở rộng, giúp Agent học hỏi và thích nghi liên tục. Bộ nhớ cho phép Agent duy trì thông tin trong thời gian dài, tích lũy kinh nghiệm sau khi triển khai, bao gồm cả thành công và thất bại, đồng thời điều chỉnh chiến lược mà không cần huấn luyện lại mô hình.

Thứ ba, **hỗ trợ khả năng tự cải thiện**. Thông qua việc tích lũy kinh nghiệm, các mẫu suy luận và phản hồi từ người dùng, bộ nhớ giúp Agent từng bước nâng cao khả năng thích nghi và hiệu suất xử lý nhiệm vụ. Nhờ đó, LLM Agent có thể cải thiện hành vi và kết quả thực hiện nhiệm vụ mà không cần cập nhật trực tiếp các tham số của mô hình.

Thứ tư, **giảm thiểu hiện tượng ảo giác**. Khi phản hồi của Agent được đặt nền tảng trên các nội dung bộ nhớ có cấu trúc và có thể kiểm chứng, hệ thống sẽ giảm sự phụ thuộc vào tri thức tham số có thể không chính xác hoặc đã lỗi thời của mô hình. Nói cách khác, bộ nhớ giúp Agent đưa ra phản hồi có căn cứ hơn, nhất quán hơn và ít bị sai lệch hơn.

Tóm lại, bộ nhớ giúp chuyển đổi các mô hình phản hồi không trạng thái thành các hệ thống thích nghi có trạng thái. Nhờ có bộ nhớ, Agent có thể xây dựng mối quan hệ dài hạn với người dùng, học hỏi từ lịch sử tương tác và ngày càng thể hiện hành vi cá nhân hóa, linh hoạt và thông minh hơn.



Hình II.1: Minh họa mối quan hệ giữa Agent và Agent Memory

2 Retrieval-Augmented Generation

Retrieval-Augmented Generation, hay RAG, là phương pháp kết hợp mô hình ngôn ngữ lớn với hệ thống truy xuất thông tin bên ngoài. Thay vì chỉ dựa vào tri thức đã học trong quá trình huấn luyện, mô hình có thể tìm kiếm các tài liệu hoặc đoạn thông tin liên quan, sau đó sử dụng chúng làm ngữ cảnh bổ sung để sinh câu trả lời.

Một hệ thống RAG cơ bản thường gồm ba bước chính. Đầu tiên, dữ liệu như tài liệu hoặc lịch sử hội thoại được chia thành các đoạn nhỏ, gọi là chunks, rồi được chuyển thành vector embedding và lưu trong cơ sở dữ liệu vector. Tiếp theo, khi người dùng đặt câu hỏi, hệ thống chuyển câu hỏi thành embedding và tìm các chunks có độ tương đồng cao nhất. Cuối cùng, các chunks được truy xuất sẽ được đưa vào prompt cùng với câu hỏi để mô hình ngôn ngữ sinh câu trả lời.

Trong LLM Agent, RAG có thể được xem như một cơ chế bộ nhớ ngoài. Các thông tin từ tài liệu, lịch sử hội thoại hoặc tri thức miền ứng dụng có thể được lưu trữ và truy xuất khi cần. Nhờ đó, Agent có thể phản hồi dựa trên cả ngữ cảnh hiện tại và thông tin đã lưu trước đó, giúp tăng tính nhất quán và khả năng cá nhân hóa.

Tuy nhiên, RAG truyền thống vẫn có một số hạn chế. Phương pháp này chủ yếu dựa vào độ tương đồng ngữ nghĩa giữa câu hỏi và từng đoạn văn bản riêng lẻ, nên có thể bỏ sót các thông tin liên quan gián tiếp. Với các câu hỏi yêu cầu suy luận nhiều bước hoặc cần tổng hợp bằng chứng từ nhiều phiên hội thoại, vector retrieval đơn thuần thường chưa đủ hiệu quả.

Vì vậy, nhiều nghiên cứu đã mở rộng RAG bằng cách kết hợp thêm Knowledge Graph, cơ chế reranking hoặc các phương pháp truy xuất có cấu trúc. Trong các hệ thống Agent Memory, hướng tiếp cận này giúp tổ chức thông tin tốt hơn, biểu diễn được quan hệ giữa

các thực thể và hỗ trợ suy luận dài hạn hiệu quả hơn.

3 Knowledge Graph trong hệ thống memory

Knowledge Graph, hay đồ thị tri thức, là cấu trúc biểu diễn thông tin dưới dạng các thực thể và quan hệ giữa các thực thể. Trong đó, mỗi node biểu diễn một thực thể như người, địa điểm, sự kiện hoặc khái niệm; mỗi edge biểu diễn quan hệ giữa hai thực thể. Một đơn vị thông tin cơ bản trong Knowledge Graph thường có dạng triple: chủ thể, quan hệ, đối tượng.

Trong Agent Memory, Knowledge Graph giúp tổ chức bộ nhớ dài hạn theo cách có cấu trúc hơn so với việc lưu trữ hội thoại dưới dạng văn bản thô. Từ lịch sử hội thoại, hệ thống có thể trích xuất các entity-relation triples và liên kết chúng với đoạn hội thoại gốc. Nhờ vậy, Agent vừa có thể suy luận trên graph, vừa có thể truy ngược lại bằng chứng ban đầu khi cần sinh câu trả lời.

So với vector database, Knowledge Graph có ưu điểm là biểu diễn rõ ràng mối quan hệ giữa các thông tin. Điều này đặc biệt hữu ích trong các câu hỏi yêu cầu suy luận nhiều bước, khi câu trả lời không nằm trong một đoạn văn bản duy nhất mà cần kết nối nhiều thực thể hoặc sự kiện khác nhau.

Tuy nhiên, Knowledge Graph trong Agent Memory cũng có một số thách thức. Quá trình trích xuất thực thể và quan hệ thường phụ thuộc vào LLM, nên có thể xuất hiện thông tin sai, thiếu hoặc nhiễu. Ngoài ra, khi lịch sử hội thoại tăng lên, graph cũng trở nên phức tạp hơn, đòi hỏi cơ chế truy xuất và xếp hạng hiệu quả.

4 Personalized PageRank

Personalized PageRank là một biến thể của thuật toán PageRank, dùng để đánh giá mức độ liên quan của các node trong graph đối với một tập node khởi đầu cụ thể. Khác với PageRank truyền thống, vốn tính độ quan trọng tổng quát của các node trên toàn đồ thị, Personalized PageRank tập trung lan truyền điểm số từ các node được chọn ban đầu, gọi là seed nodes.

Ý tưởng chính của Personalized PageRank là mô phỏng quá trình di chuyển ngẫu nhiên trên graph. Tại mỗi bước, thuật toán có thể đi theo các cạnh để chuyển sang node lân cận hoặc quay trở lại các seed nodes với một xác suất nhất định. Nhờ đó, các node nằm gần seed nodes hoặc có nhiều liên kết liên quan đến seed nodes sẽ nhận được điểm số cao hơn.

Công thức tổng quát của Personalized PageRank được biểu diễn như sau:

$$\mathbf{p}^{(t+1)} = \alpha \mathbf{M} \mathbf{p}^{(t)} + (1 - \alpha) \mathbf{v}$$

Trong đó:

$$\mathbf{p}^{(t)}$$

là vector điểm PageRank tại bước lặp thứ t .

$$\mathbf{M}$$

là ma trận chuyển trạng thái của graph, thể hiện xác suất di chuyển từ node này sang node khác.

$$\alpha$$

là hệ số damping factor, thường nằm trong khoảng từ 0 đến 1. Hệ số này quyết định xác suất tiếp tục di chuyển theo các cạnh trong graph.

$$\mathbf{v}$$

là vector cá nhân hóa, trong đó các seed nodes được gán xác suất ban đầu cao hơn các node khác. Nếu có k seed nodes, mỗi seed node thường được gán giá trị:

$$\frac{1}{k}$$

còn các node không thuộc seed nodes được gán giá trị bằng 0.

Quá trình lặp được thực hiện cho đến khi vector $\mathbf{p}$ hội tụ. Khi đó, giá trị của mỗi phần tử trong $\mathbf{p}$ thể hiện mức độ liên quan của node tương ứng đối với tập seed nodes ban đầu.

Trong Agent Memory, Personalized PageRank có thể được dùng để truy xuất thông tin trên Knowledge Graph. Khi người dùng đặt câu hỏi, hệ thống xác định các thực thể liên quan trong query và sử dụng chúng làm seed nodes. Sau đó, thuật toán được chạy trên entity-relation graph để tìm ra các thực thể, quan hệ hoặc triples có liên quan trực tiếp và gián tiếp đến câu hỏi.

Ưu điểm của Personalized PageRank là khả năng mở rộng truy xuất vượt ra ngoài matching trực tiếp. Thay vì chỉ tìm các triples có độ tương đồng cao với query, hệ thống có thể khai thác cấu trúc liên kết của graph để phát hiện các thông tin nằm trong vùng lân cận ngữ nghĩa. Điều này đặc biệt hữu ích với các câu hỏi cần suy luận nhiều bước hoặc cần kết nối nhiều bằng chứng từ các phiên hội thoại khác nhau.

Chương III

Tổng quan các nghiên cứu liên quan

Chương IV

Phân tích bài toán và thiết kế phương pháp đề xuất

Chương V

Thực nghiệm và đánh giá

Chương VI

Kết luận và hướng phát triển